



Guião 1 – Classes e Objectos

Objectivos:

- Definição de classes
 - Inicialização
 - Atributos
 - Métodos
- Instânciação de Objectos
 - id
 - Referências

1. Numa *interactive shell* Python 3 defina uma classe **Pessoa**.

Obs.: Se ainda não tem o Python instalado, pode utilizar esta shell online:
<https://www.python.org/shell/>

2. Instancie uma classe **Pessoa** e faça a atribuição a uma variável **pessoa_1**.

Interprete o que vê quando executa:

```
>>>pessoa_1
```

3. Verifique o id do objecto referenciado por **pessoa_1**.

Obs.: Use a função `id()`. Não sabes como utilizar a função `id()`?

```
>>>help(id)
```

4. Compare os valores obtidos no ponto 2 e 3.

5. Repita o ponto 2 e reveja o id. O que alterou?

6. A função `sys.getrefcount()` permite saber quantas referências estão activas para um determinado objecto. Veja quantas referências existem para o objecto referenciado pela variável **pessoa_1**. Comente o resultado.



7. Analise o código

```
class Pessoa():  
    pass  
  
pessoa_1 = Pessoa()  
pessoa_2 = Pessoa()  
  
a = pessoa_1  
  
lista = []  
lista.append(a)
```

Quantas referencias estarão activas para a o objecto referenciado pela variável pessoa_1?

8. Analise, teste e comente:

```
a=1  
b=1  
print(a is b)  
  
a=[1]  
b=[1]  
print(a is b)  
  
a=[1]  
b=a  
print(a is b)  
  
a=[1]  
b=a+[] #ou b=a[:]  
print(a is b)
```

9. Implemente a classe

```
class PizzaMedia():  
    diametro_cm=30  
    def __init__(self, nome):  
        self.nome = nome
```

Documente quanto aos atributos desta classe.



10. Adicione um método que devolva o nome em MAIÚSCULAS.

12. Adicione um atributo que inclua os ingredientes da pizza sob a forma de uma lista com str. Não esquecer da inicialização.

13. Garanta que quando os ingredientes são omitidos, a pizza fica com ‘tomate’ e ‘queijo mozzarella’.

14. Crie um método que devolva a área da pizza. Que tipo de método foi mais conveniente?

15. Imagina agora que além de pizzas médias também queres disponibilizar pizzas pequenas e grandes.

16. Documente devidamente a classe e todos os métodos.

17. Finalize o guião e envie via moodle.

Classes e Objetos

1.

```
ines@ines:~/Documents/PA_ProgramacaoAplicada/Semana4$ python3
Python 3.12.3 (main, Jan 22 2026, 20:57:42) [GCC 13.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
>>>
>>> █
```

2.

```
>>> class Pessoa():
...     pass
...
>>>
>>> pessoa_1 = Pessoa()
>>>
>>> pessoa_1
<__main__.Pessoa object at 0x707af98b8c80>
```

O valor 0x 707af98b8c80 indica a posição de memória em que pessoa_1 (objeto da classe Pessoa) se encontra, em hexadecimal (base 16).

3.

```
>>> id(pessoa_1)
123673474993280
```

O valor 123673474993280 representa a posição de memória onde se encontra a variável pessoa_1, em decimal. É um valor único.

4. O valor em 2. indica a posição em memória em que pessoa_1 se encontra, e o valor está representado em hexadecimal. O valor em 3. corresponde ao mesmo valor, mas em decimal.

5.

```
>>> pessoa_1 = Pessoa()
>>> pessoa_1
<__main__.Pessoa object at 0x707af975fe00>
>>> id(pessoa_1)
123673473580544
```

Criação de um novo objeto pessoa_1 do tipo Pessoa, valor do id altera-se.

6.

```
>>> import sys
>>>
>>> sys.getrefcount(pessoa_1)
2
```

Existe uma referência para o objeto referenciado pela variável `pessoa_1`. A outra referência diz respeito à chamada da função `getrefcount`, que cria sempre uma referência temporária.

7. 3 referências para `pessoa_1` (mais uma extra, que diz respeito à chamada da função `getrefcount()`).

`pessoa_1`, `a`, `lista[0]`.

```
ines@ines:~/Documents/PA_ProgramacaoAplicada/Semana4$ python3 ex7.py
4
```

8.

```
ines@ines:~/Documents/PA_ProgramacaoAplicada/Semana4$ python3 ex8.py
True
False
True
False
```

```
a=1
b=1
print(a is b)
```

1º: True, porque `a` e `b` apontam para o mesmo objeto.

```
a=[1]
b=[1]
print(a is b)
```

2º: False, porque `a[0]` e `b[0]` são objetos independentes, e apontam para diferentes posições em memória.

```
a=[1]
b=a
print(a is b)
```

3º: True, porque b aponta para a[0], e a aponta para a[0], portanto são referências que apontam para o mesmo objeto.

```
a=[1]
b=a+[] #ou b=a[:]
print(a is b)
```

4º: False, porque a devolve a referência para a[0], e b corresponde à referência para b[0].

a e b correspondem a ids diferentes pois são listas diferentes.

9.

```
class PizzaMedia():
    """ cada PizzaMedia criada tem 30cm de diâmetro """
    diametro_cm=30

    def __init__(self, nome):
        self.nome = nome
```

10.

```
class PizzaMedia():
    """ cada PizzaMedia criada tem 30cm de diâmetro """
    diametro_cm=30

    def __init__(self, nome):
        self.nome = nome

    def get_upper(self):
        return self.nome.upper()
```

12.

```
def __init__(self, nome, pizza_ingredients ):
    self.nome = nome
    self.ingredients = pizza_ingredients
```

```
def __init__(self, nome, pizza_ingredients):
    self.nome = nome
    self.ingredients = pizza_ingredients

def get_upper(self):
    return self.nome.upper()

pizza_vegetariana = PizzaMedia("Vegetariana", ["queijo", "pimentos", "orégãos", "milho" ] )
```

13.

```
def __init__(self, nome, pizza_ingredients = ["tomate", "queijo mozzarella" ]):
    self.nome = nome
    self.ingredients = pizza_ingredients
```

tomate e queijo mozzarella ficam como ingredientes default de qualquer PizzaMedia instanciada.

14.

```
def pizza_area(self):
    return math.pi * ((self.diametro_cm/2)**2)
```

Um método de instância (chamado a partir de um objeto, método que atua sobre os atributos de uma instância de uma classe, através do *self*) foi mais conveniente, pois depende do diâmetro da pizza.

15.

```
import math

class PizzaPequena():
    """ cada PizzaPequena criada tem 20cm de diâmetro """
    diametro_cm=20

    def __init__(self, nome, pizza_ingredients = ["tomate", "queijo mozzarella" ]):
        self.nome = nome
        self.ingredients = pizza_ingredients

    def get_upper(self):
        return self.nome.upper()

    def pizza_area(self):
        return math.pi * ((self.diametro_cm/2)**2)
```

```

import math

class PizzaGrande():
    """ cada PizzaGrande criada tem 50cm de diâmetro """
    diametro_cm=50

    def __init__(self, nome, pizza_ingredients = ["tomate", "queijo mozzarella"] ):
        self.nome = nome
        self.ingredients = pizza_ingredients

    def get_upper(self):
        return self.nome.upper()

    def pizza_area(self):
        return math.pi * ((self.diametro_cm/2)**2)

```

16.

```

import math

class PizzaMedia():
    """ PizzaMedia encapsula uma pizza com 30cm de diâmetro, que tem um
    nome e uma lista de ingredientes como atributos"""

    diametro_cm=30

    """
    Inicializa um novo objeto PizzaMedia
    :param nome: nome da pizza
    :param pizzaingredients: lista de ingredientes da pizza (cada ingrediente é uma str)
    :returns nothing
    """

    def __init__(self, nome, pizza_ingredients = ["tomate", "queijo mozzarella"] ):
        self.nome = nome
        self.ingredients = pizza_ingredients

    """
    Retorna o nome da pizza em maiúsculas.
    """
    def get_upper(self):
        return self.nome.upper()

    """
    Calcula e devolve a área da pizza, em centímetros quadrados.
    """
    def pizza_area(self):
        return math.pi * ((self.diametro_cm/2)**2)

#help(PizzaMedia)

pizza vegetariana = PizzaMedia("Vegetariana",["queijo", "pimentos","orégãos", "milho" ] )
print(f"Ingredientes desta Pizza {pizza_vegetariana.nome}: {pizza_vegetariana.ingredients}")

```